Web Engineering & Development (SWE 363)

Interactive Front-End Development

In today's lecture:

- Styling
- React Bootstrap
- Fetching Data

Reference:

- Zybook: 5.15-5.17

5.15 Styling (React)

What is CSS Styling in React?

CSS = Cascading Style Sheets

- Controls the **visual appearance** of your React components
- Colors, fonts, spacing, layout, animations
- Makes your app look **professional** and **user-friendly**

Why style React components?

- Default HTML elements look **boring**
- Good styling = **better user experience**
- Professional appearance = **user trust**

Three Ways to Style React Components

1. Inline Styles

```
<div style={{
  backgroundColor: 'blue',
  color: 'white',
  padding: '20px'
}}>
  Hello World
</div>
```

Three Ways to Style React Components

2. CSS Stylesheets

```
// In your component
import "./Welcome.css";

<div className="my-card">Hello World</div>

// In your CSS file
.my-card {
  background-color: blue;
  color: white;
  padding: 20px;
}
```

Three Ways to Style React Components

3. CSS Modules (Advanced)

```
// In your component

import styles from './MyComponent.module.css';
<div className={styles.card}>Hello World</div>

// In your MyComponent.module.css file
.card{
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 10px;
}
```

CSS Properties You Need to Know

Layout Properties

```css
.container {
  width: 100%;              /* Full width */
  height: 200px;            /* Fixed height */
  margin: 10px;             /* Space outside */
  padding: 15px;            /* Space inside */
  display: flex;            /* Flexbox layout */
}
```

CSS Properties You Need to Know

Visual Properties

```css
.card {
  background-color: #ffffff;  /* Background color */
  color: #333333;             /* Text color */
  border: 1px solid #ccc;     /* Border */
  border-radius: 8px;         /* Rounded corners */
  box-shadow: 0 2px 4px rgba(0,0,0,0.1); /* Shadow */
}
```

CSS Variables (Custom Properties)

Define once, use everywhere!

```css
:root {
  --primary-color: #0d6efd;
  --secondary-color: #6c757d;
  --light-color: #f8f9fa;
  --dark-color: #212529;
}
.my-button {
  background-color: var(--primary-color);
  color: white;
}
.my-card {
  background-color: var(--light-color);
  border: 1px solid var(--secondary-color);
}
```

CSS Variables (Custom Properties)

```css
:root {
  --primary-color: #0d6efd;
  --secondary-color: #6c757d;
  --light-color: #f8f9fa;
  --dark-color: #212529;
}
```

Benefits:

- **Consistent** colors across your app
- **Easy** to change theme
- **Maintainable** code

CSS Responsive Design

Make your app work on all devices!

```css
.user-avatar { /* Mobile first approach */
  width: 50px;
  height: 50px;
}
@media (min-width: 768px) { /* Tablet and up */
  .user-avatar {
    width: 60px;
    height: 60px;
  }
}
@media (min-width: 1024px) {/* Desktop and up */
  .user-avatar {
    width: 80px;
    height: 80px;
  }
}
```

5.16 React Bootstrap

What is Bootstrap?

Bootstrap = CSS framework

- **Pre-built** components and styles
- **Responsive** design out of the box
- **Consistent** look across browsers
- **Saves time** - no need to write CSS from scratch

React-Bootstrap = Bootstrap components as React components

- Same styling, but **React-friendly**
- **Props** instead of classes
- **Better integration** with React

React Bootstrap

React Bootstrap Website

Installing React Bootstrap

```
# Install React Bootstrap and Bootstrap
npm install react-bootstrap bootstrap

# Add Bootstrap CSS to your index.html
```

```
<!-- In public/index.html -->
<link
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
  rel="stylesheet"
>
```

Common Bootstrap Components

Layout Components

```
import { Container, Row, Col } from 'react-bootstrap';

<Container>
  <Row>
    <Col md={6} lg={4}>
      <h3>Column 1</h3>
    </Col>
    <Col md={6} lg={4}>
      <h3>Column 2</h3>
    </Col>
    <Col md={12} lg={4}>
      <h3>Column 3</h3>
    </Col>
  </Row>
</Container>
```

Bootstrap Utility Classes

Quick styling without writing CSS!

Spacing

```
<div className="mb-4">      {/* margin-bottom: 1.5rem */}
<div className="py-3">      {/* padding top & bottom: 1rem */}
<div className="px-2">      {/* padding left & right: 0.5rem */}
<div className="mt-5">      {/* margin-top: 3rem */}
```

Bootstrap Utility Classes

Colors & Backgrounds

```
<div className="bg-primary text-white">  {/* Blue background, white text */}
<div className="bg-light text-dark">    {/* Light background, dark text */}
<div className="text-center">           {/* Center align text */}
<div className="shadow">                {/* Add shadow */}
```

Bootstrap Utility Classes

Display & Flexbox

```
<div className="d-flex">                    {/* display: flex */}
<div className="justify-content-center"> {/* justify-content: center */}
<div className="align-items-center">    {/* align-items: center */}
```

5.17 Fetching Data

Why Fetch Data in React?

Real apps need real data!

- Display **user information** from a database
- Show **products** from an online store
- Load **posts** from social media
- Get **weather** information

Why Fetch Data in React?

Without data fetching:

- Static, boring apps
- No real-world functionality

With data fetching:

- Dynamic, interactive apps
- Real-world applications

What is an API?

API = Application Programming Interface

- A way for your app to **talk to servers**
- **Request** data from a server
- **Receive** data back
- Like ordering food from a restaurant!

Common APIs:

- `https://jsonplaceholder.typicode.com/users` - Fake user data
- `https://api.github.com/users` - GitHub user data
- `https://api.openweathermap.org/data/2.5/weather` - Weather data

How to Fetch Data from an API

JavaScript's built-in way to get data

```
// Basic fetch
fetch('https://jsonplaceholder.typicode.com/users')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

How to Fetch Data from an API

With async/await (cleaner syntax):

```
const fetchUsers = async () => {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/users');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error:', error);
  }
};
```

React Hooks for Data Fetching

useState - Store the data

```
import { useState } from 'react';

const [users, setUsers] = useState([]);          // Store user data
const [loading, setLoading] = useState(true);  // Loading state
const [error, setError] = useState(null);       // Error state
```

React Hooks for Data Fetching

useEffect - Fetch data when component loads

```
import { useState, useEffect } from 'react';

useEffect(() => {
  const fetchUsers = async () => {
    try {
      setLoading(true);
      const response = await fetch('https://jsonplaceholder.typicode.com/users');
      const data = await response.json();
      setUsers(data);
    } catch (err) { setError(err.message);}
      finally { setLoading(false); }
  };
  fetchUsers();
}, []); // Empty array = run once when component mounts
```

React Hooks for Data Fetching

Loading States & Error Handling

Show feedback to users!

Loading State

```
{loading && (
  <div className="text-center">
    <Spinner animation="border" />
    <p>Loading users...</p>
  </div>
)}
```

React Hooks for Data Fetching

Loading States & Error Handling

Show feedback to users!

Error State

```
{error && (
  <Alert variant="danger">
    <Alert.Heading>Error!</Alert.Heading>
    <p>{error}</p>
  </Alert>
)}
```

React Hooks for Data Fetching

Loading States & Error Handling

Show feedback to users!

Empty State

```
{users.length === 0 && !loading && (
  <Alert variant="info">
    No users found.
  </Alert>
)}
```

30m

Demo

5.6 React Styling & Fetching

Next Class

- Quiz 2